PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING

COURSE: SOFTWARE ARCHITECTURE
HOTEL MANAGEMENT SYSTEM

INSTRUCTOR: Vũ Quang Dũng

GROUP 7: Trần Ngọc An (23010283 – K17-KTPM)

CREDIT CLASS: CSE703110-1-2-25(N01)

Ha Noi, March 6, 2026

Contents

1. Executive Summary

This project implements a Hotel Booking Management System that supports the end-to-end workflow of a real hotel: customers can browse rooms, create bookings, make payments, manage their wallet, and view booking history; receptionists can check guests in and out and manage service requests; administrators can manage rooms, services, staff, coupons, and reports. The system is architected using a layered architecture on top of Flask, with clear separation between controllers (HTTP layer), services (business logic), repositories (data access), and models (domain entities).

The backend lives under SRC/Arch and exposes REST-style JSON APIs (e.g. /api/bookings, /api/rooms, /api/auth/login, /api/wallet) through controller classes such as booking_controller.py, room_controller.py, user_controller.py, payment_controller.py, coupon_controller.py, checkin_controller.py, staff_controller.py, report_controller.py, and wallet_controller.py. These controllers rely on service classes (e.g. booking_service.py, room_service.py, payment_service.py, coupon_service.py) that encapsulate business logic and call repository classes (e.g. booking_repository.py, room_repository.py, coupon_repository.py) to access a MySQL database.

The frontend is implemented in SRC/UI as a set of HTML/CSS/JS pages for different roles: public pages for browsing rooms, authentication pages for login and registration, user pages for dashboard, my bookings, payment, and wallet, and admin pages for room and staff management and reports. The final outcome is a working multi-role hotel management application where the architecture supports modifiability, maintainability, and basic security and scalability for small-to-medium usage.

2. Architectural Design & Implementation

2.1 Architectural Pattern & Layers

The backend follows a layered architecture on top of Flask:

- Presentation/API Layer – Controllers (SRC/Arch/controllers): booking_controller.py, room_controller.py, user_controller.py, service_controller.py, payment_controller.py, coupon_controller.py, checkin_controller.py, staff_controller.py, report_controller.py, wallet_controller.py.

These classes receive HTTP requests (via routes defined in app.py), parse and validate incoming JSON, call appropriate service methods, and use View classes (e.g. booking_view.py) to format the JSON responses.

- Business Layer – Services (SRC/Arch/services):
  + booking_service.py handles booking lifecycle: create_booking, update_booking, cancel_booking, get_user_bookings. It validates input, checks dates, consults room_service.py for availability, interacts with room_repository.py to reserve rooms, and optionally triggers payment_service.py for immediate payment.
  + coupon_service.py validates coupon creation and apply_coupon logic, enforcing rules on discount types, values, and validity dates.
  + Additional services such as room_service.py, payment_service.py, checkin_service.py (if present), report_service.py, staff_service.py, user_service.py, and wallet_service.py encapsulate domain logic for their respective areas.

- Data Access Layer – Repositories (SRC/Arch/repository):
  + booking_repository.py implements CRUD operations on the bookings table using mysql.connector, with methods like save, find_by_id, find_all, find_by_user_id, find_by_room_id, find_available_rooms_by_date_range, update, and delete. The method find_available_rooms_by_date_range includes SQL logic to exclude rooms with overlapping active bookings.
  + Similar repository classes exist for rooms, check-ins, coupons, payments, service requests, staff, users, and wallets.

- Domain Model Layer – Models (SRC/Arch/models)
  + Classes such as booking.py, room.py, user.py, staff.py, service.py, coupon.py, checkin.py, payment.py, wallet.py, report.py represent database entities in Python.
  + Booking model instances are constructed from DB rows in booking_repository.py/BookingRepository._row_to_booking.

- Middleware and Utilities
  + middleware/auth.py provides decorators (require_auth, require_admin, require_receptionist_or_admin, require_role, optional_auth) that wrap endpoints to enforce authentication and authorization.
  + utils/image_handler.py handles image uploads for room images and returns file paths in SRC/uploads/rooms, which are later served by the /uploads route in app.py.

The frontend under SRC/UI is structured by roles and pages (admin, auth, public, user), and communicates with the backend via REST APIs using JavaScript.

C4 Level 1

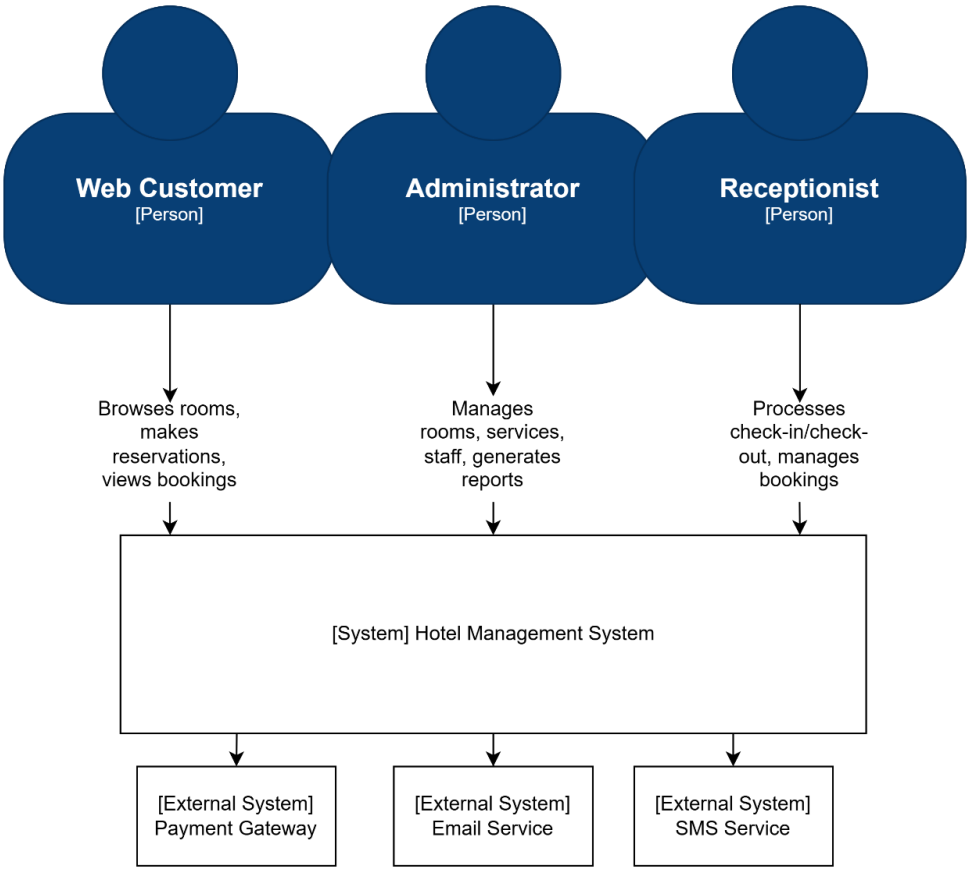

Figure 2: C4 lv 1 Diagram

At the center of the model is the Hotel Management System, a single Flask-based monolithic backend with a static web UI. Around this system boundary are three primary human actors:

- Web Customer: browses rooms, creates and manages bookings, performs payments, manages wallet balance, and submits service requests or contact messages.
- Receptionist: manages daily hotel operations such as check-in/check-out, walk-in bookings, room assignment, and monitoring of transactions and service requests.
- Administrator: manages room inventory, services, staff accounts, user accounts, and analytical reports.

All interactions from these actors go through the web browser UI into the monolithic backend, which in turn persists state in a single MySQL database. External services such as third-party payment gateways or email providers are considered future integrations but are not part of the current implementation.

C4 Level 2 - Container Diagram

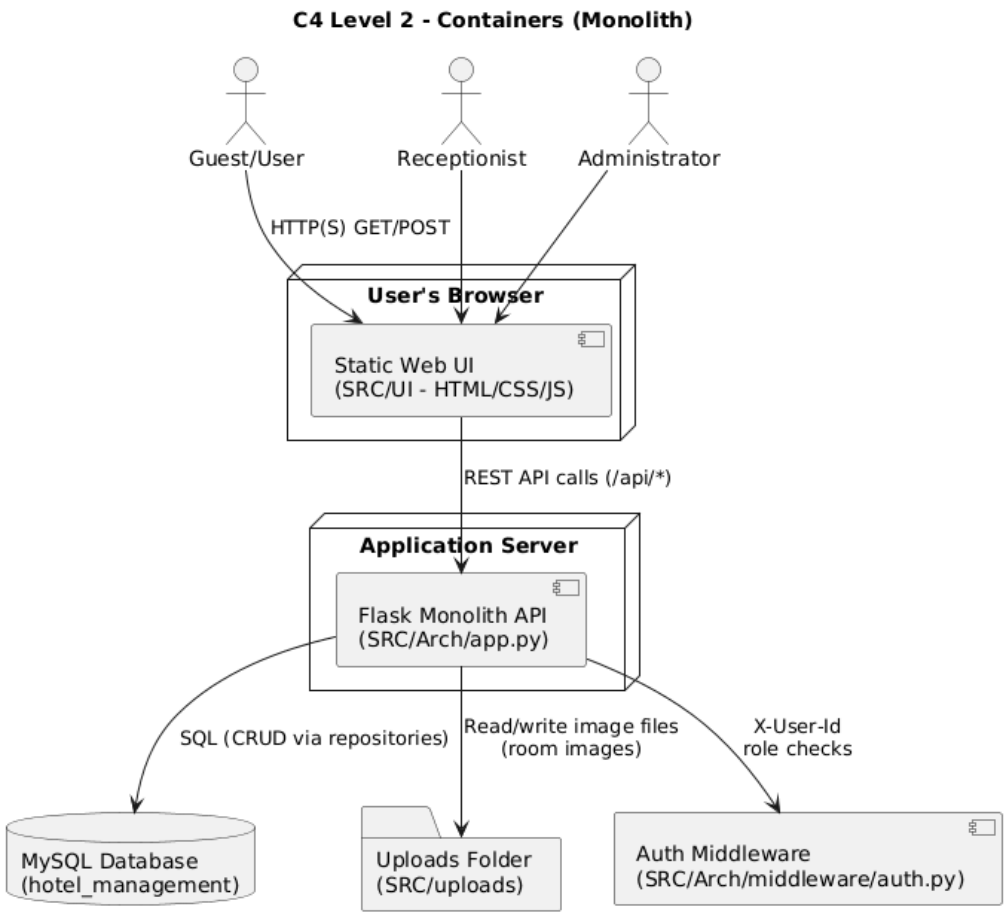

Figure 3: C4 lv 2 - Container Diagram

At the container level, the Hotel Management System is decomposed into three main containers:

- Web Browser (Static Web UI – SRC/UI): HTML/CSS/JavaScript pages grouped by role (public, auth, user, admin, receptionist). JavaScript (notably api.js) calls backend APIs under /api/ and renders booking, wallet, payment, and management screens.
- Flask Monolith API (SRC/Arch): a single Flask application (app.py) exposing REST-style JSON endpoints for rooms, bookings, payments, coupons, wallet, service requests, staff, reports, and authentication. It implements all business rules and orchestrates database access.
- MySQL Database: the relational store for users, rooms, bookings, payments, wallets, coupons, service requests, staff, and reports, configured via SRC/Arch/db_config.py.

Static file storage for uploaded room images (SRC/uploads/rooms) is also modeled as an infrastructure container, served via the /uploads/<filename> route from the Flask monolith. The browser communicates only with the Flask API and /uploads/; the API is the single integration point to MySQL and the uploads storage.

C4 Level 3 - Components Diagram

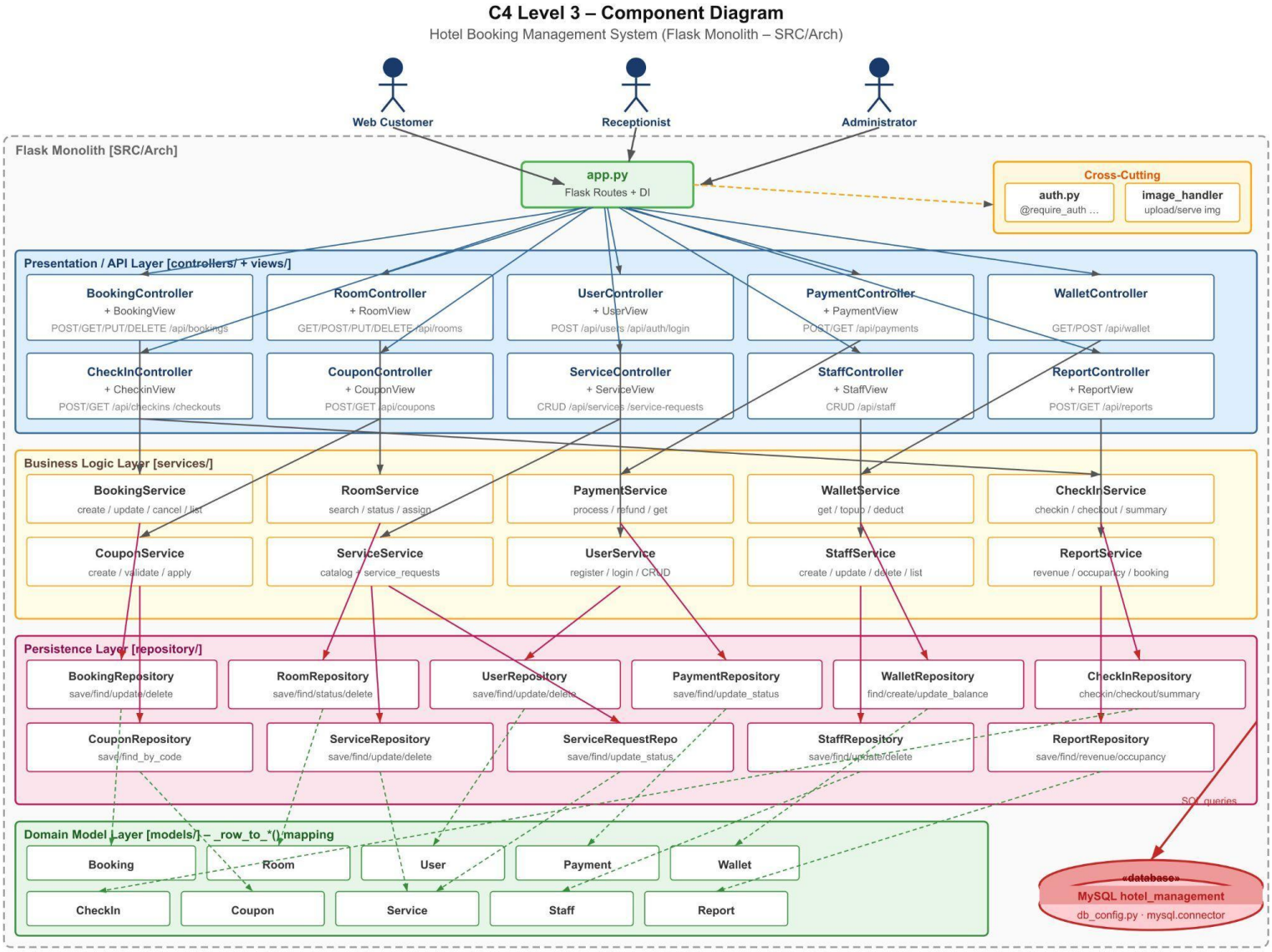

Figure 4: C4 lv 3 - Components Diagram

Inside the Flask monolith, the main components are organized into layers:

- Routing / Entry Point: app.py defines Flask routes and wires them to controller methods. It also applies authentication/authorization decorators from middleware/auth.py.
- Controllers (SRC/Arch/controllers): one controller per domain (e.g., booking_controller.py, room_controller.py, user_controller.py, payment_controller.py, coupon_controller.py, checkin_controller.py, staff_controller.py, report_controller.py, wallet_controller.py, service_controller.py). Controllers parse HTTP requests, perform basic validation, and delegate to services.

- Views (SRC/Arch/views): view classes (e.g., booking_view.py, room_view.py, payment_view.py) format domain objects and errors into consistent JSON responses.
- Services (SRC/Arch/services): business-logic components such as booking_service.py, room_service.py, payment_service.py, wallet_service.py, coupon_service.py, service_service.py, checkin_service.py, staff_service.py, report_service.py, and user_service.py. They implement booking lifecycle, availability checks, payment and refund rules, wallet updates, coupon validation, reporting, and staff/user/service management.
- Repositories (SRC/Arch/repository): data-access components (booking_repository.py, room_repository.py, user_repository.py, payment_repository.py, wallet_repository.py, coupon_repository.py, service_repository.py, service_request_repository.py, staff_repository.py, checkin_repository.py, report_repository.py) wrapping SQL queries against MySQL.
- Domain Models (SRC/Arch/models): entity classes such as booking.py, room.py, user.py, staff.py, service.py, coupon.py, checkin.py, payment.py, wallet.py, and report.py.
- Cross-cutting components: authentication/authorization middleware in middleware/auth.py (decorators like require_auth, require_admin, require_receptionist_or_admin) and the image upload handler in utils/image_handler.py used by room management.

C4 Level 4 - Code view Diagram

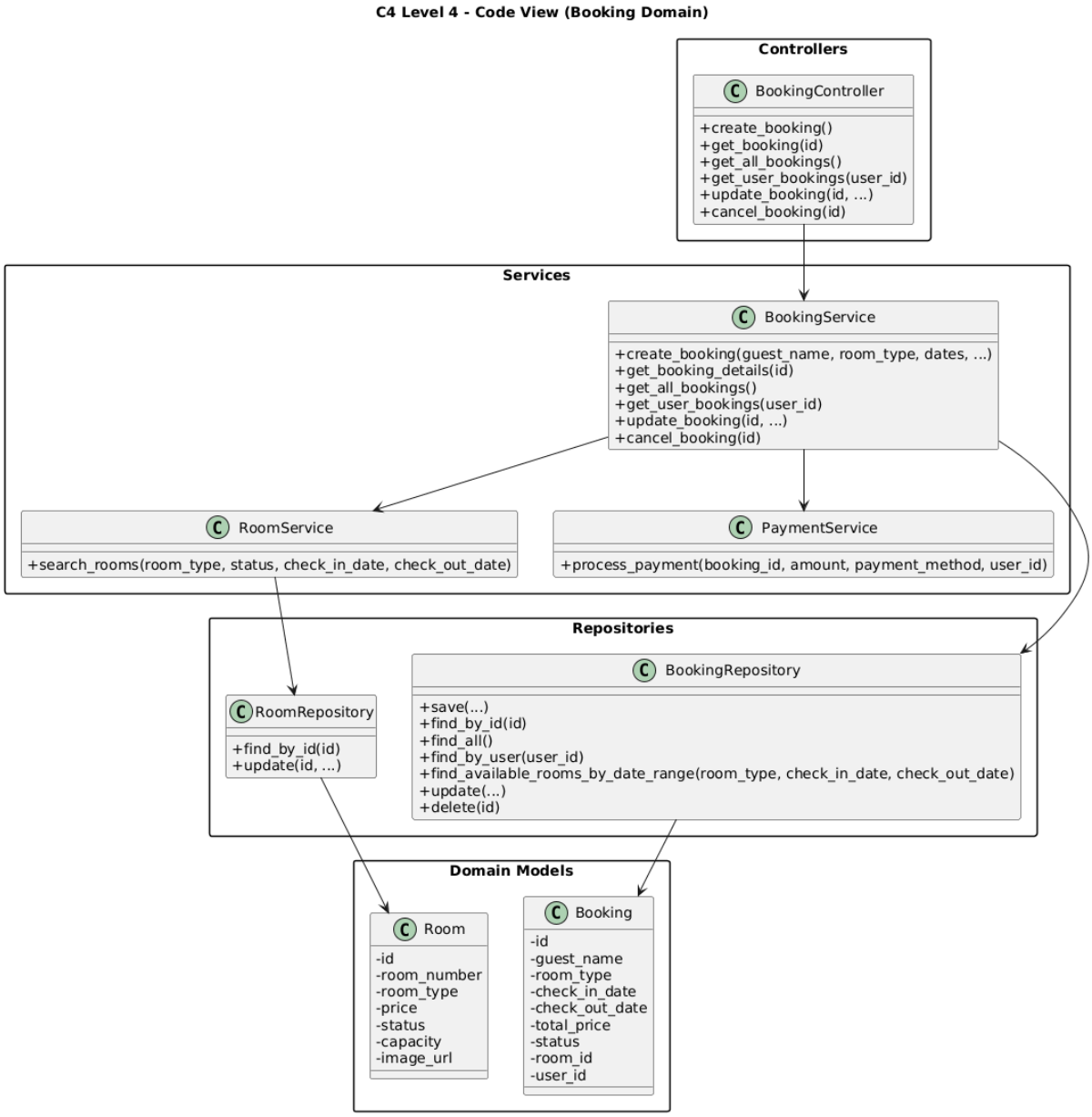

Figure 5: C4 lv 4 - Code view booking domain Diagram

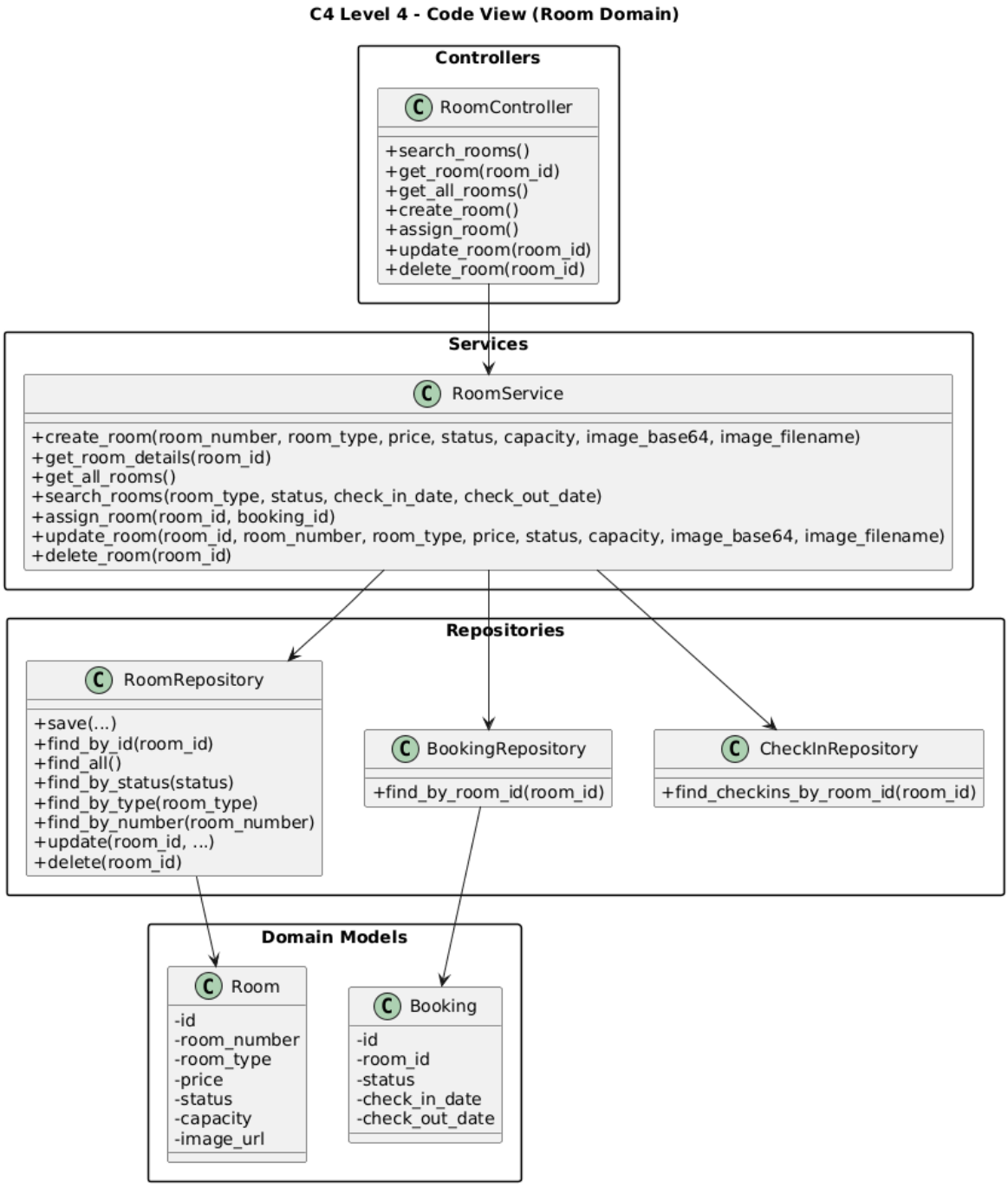

Figure 6: C4 lv 4 - Code view room domain Diagram

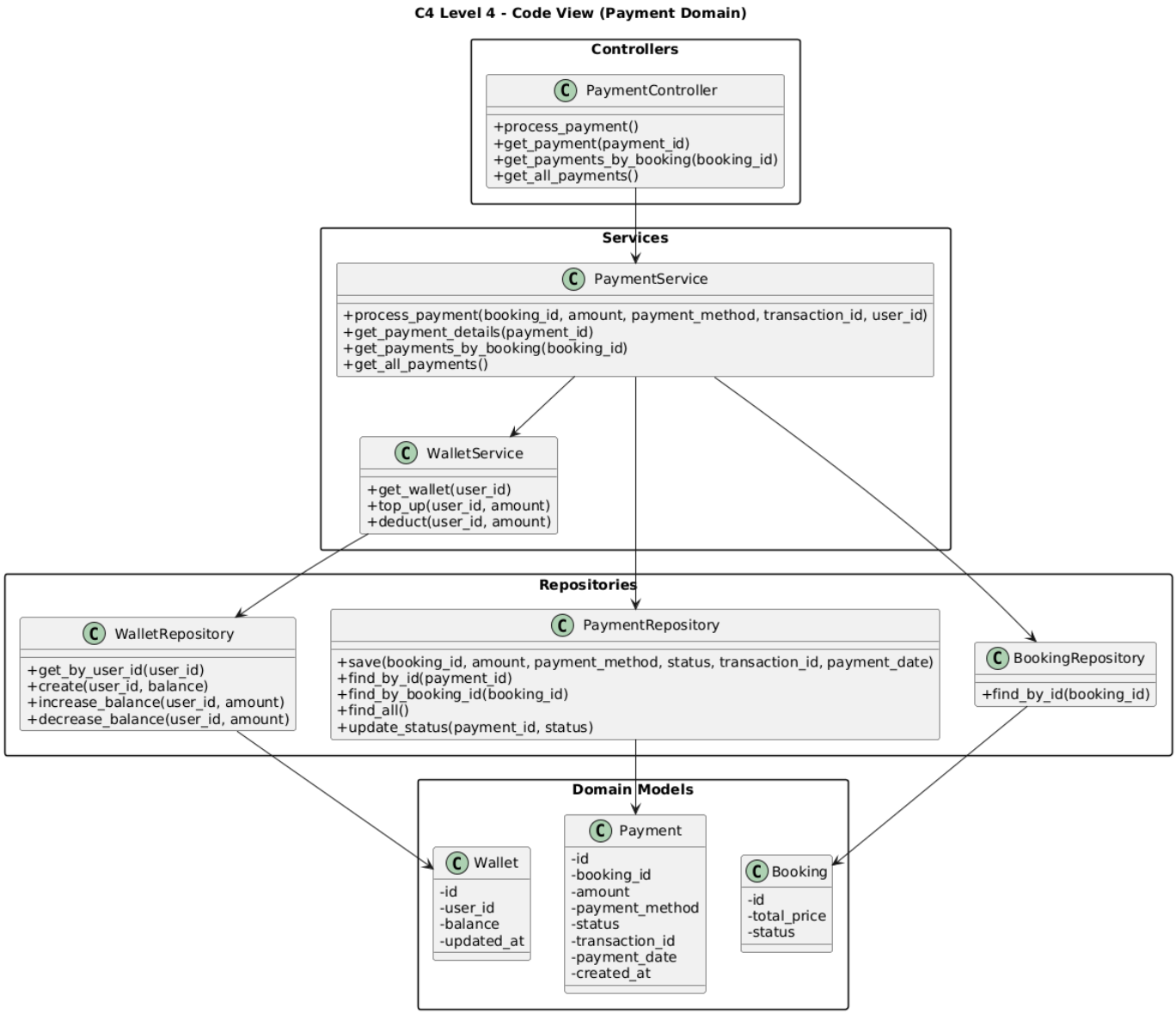

Figure 7: C4 lv 4 - Code view payment domain Diagram

At the code level, each component family is realized by a consistent set of Python modules and classes:

- Routes and composition: SRC/Arch/app.py imports all controllers, registers Flask routes (e.g., /api/bookings, /api/rooms, /api/payments), and applies middleware decorators. This file is the concrete entry point shown in the code-level diagram.
- Domain slices: for each domain (booking, room, payment, wallet, coupon, service requests, staff, reports, users, check-in/out) there is a cohesive group of modules following the pattern: controllers/<domain>_controller.py ↔ services/<domain>_service.py ↔ repository/<domain>_repository.py ↔ models/<Domain>.py ↔ views/<domain>_view.py. For example, booking_controller.py, booking_service.py, booking_repository.py, booking.py,

and booking_view.py together implement the booking slice.
- Infrastructure code: db_config.py configures MySQL connections; middleware/auth.py implements authentication and RBAC; utils/image_handler.py handles file uploads; SRC/UI/js/api.js encapsulates frontend HTTP calls to the backend.

This code-level perspective makes it clear how the high-level C4 components map directly to concrete packages, modules, and classes in the repository, and supports the architectural goals described in the ASRs (modifiability, maintainability, performance, and security).

2.2 Technical Stack and Data Model

Technical Stack:
- Backend Framework: Flask (Python) – used to define routes in app.py and integrate controllers, services, and middleware.
- CORS: flask_cors.CORS configured to allow frontend (likely served from a different origin) to access /api/ and /uploads/.
- Database: MySQL, configured via SRC/Arch/db_config.py. The DatabaseConfig class uses environment variables (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD) loaded from .env to create connections using mysql.connector.
- Data Access Pattern: Repositories use explicit SQL queries to interact with tables. For example, BookingRepository.save inserts into bookings and returns a Booking object.
- Frontend: Static HTML/CSS/JS in SRC/UI (index.html, rooms.html, room-detail.html, auth/login.html, auth/register.html, user/dashboard.html, user/my-bookings.html, user/payment.html, user/wallet.html, admin/admin.html, admin/receptionist.html, etc.).
- Static & Uploaded Content: SRC/UI/images contains UI images; SRC/uploads/rooms stores uploaded room images. app.py serves uploaded files via the /uploads/<path:filename> route (e.g., /uploads/rooms/<image>.jpg)..

Data Model:
- Room: id, room_number, room_type, price, status, capacity, image_url.
- Booking: id, user_id, guest_name, room_type, room_id, check_in_date, check_out_date, total_price, status, created_at, updated_at.
- User: id, name, email, password_hash, role (user, receptionist, admin), etc.
- Payment: id, booking_id, user_id, amount, payment_method (cash, wallet, etc.), status (pending, completed, refunded), created_at.
- Wallet: id, user_id, balance, created_at, updated_at.
- Coupon: id, code, discount_type (percentage/fixed_amount), discount_value, valid_from, valid_to, is_active.
- CheckIn: id, booking_id, room_id, check_in_time, check_out_time, status.
- Report: id, type (revenue/occupancy/booking), parameters (e.g., date range),

generated_at, content/summary fields.

2.2.1 Data Relationships

ER Diagram

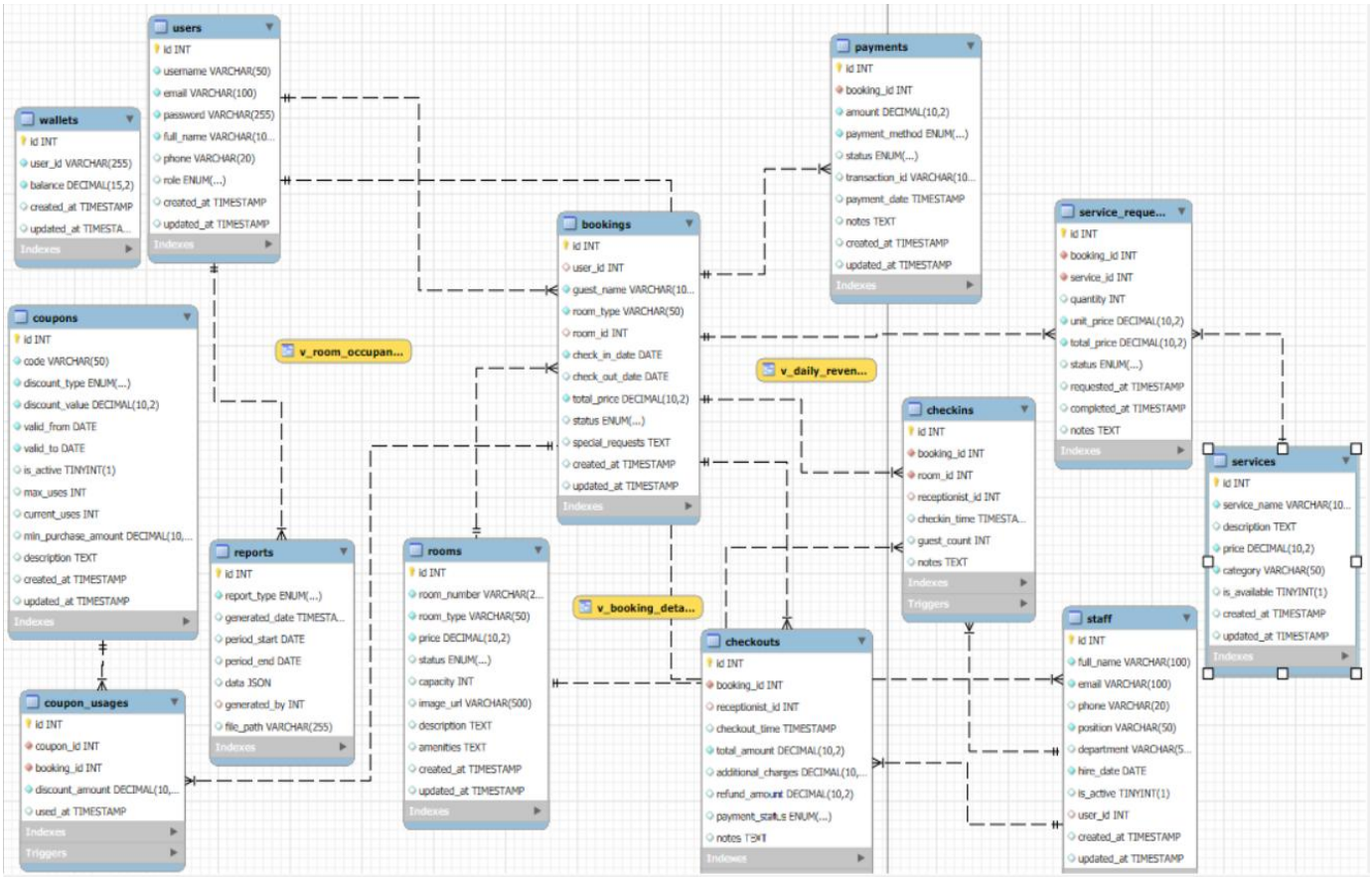

Figure 8: ER Diagram

Users table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
US1	users	id	User auto-increment ID	Y	int	11	Y
US2	users	username	Login ID	Y	varchar	50	Y
US3	users	email	User's Email	Y	varchar	100	Y
US4	users	password	Encrypted password	N	varchar	255	Y
US5	users	full_name	User's full name	N	varchar	100	Y
US6	users	phone	User's phone number	N	varchar	20	N

US7	users	role	User role	N	enum	–	Y
US8	users	created_at	Created at	N	timestamp	–	Y
US9	users	updated_at	Updated at	N	timestamp	–	Y

Staff table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
ST1	staff	id	Staff ID	Y	int	11	Y
ST2	staff	full_name	Staff's full name	N	varchar	100	Y
ST3	staff	email	User's email	Y	varchar	100	Y
ST4	staff	phone	Phone number	N	varchar	20	N
ST5	staff	position	position	N	varchar	50	Y
ST6	staff	department	department	N	varchar	50	N
ST7	staff	hire_date	Hire date	N	date	–	Y
ST8	staff	is_active	Working Status	N	boolean	–	Y
ST9	staff	user_id	FK → users.id	N	int	11	N
ST10	staff	created_at	Created at	N	timestamp	–	Y
ST11	staff	updated_at	Updated at	N	timestamp	–	Y

Rooms table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
RM1	rooms	id	Room ID	Y	int	11	Y
RM2	rooms	room_number	Room number	Y	varchar	20	Y
RM3	rooms	room_type	Room type	N	varchar	50	Y
RM4	rooms	price	Room price	N	decimal	10,2	Y
RM5	rooms	status	Room status	N	enum	–	Y
RM6	rooms	capacity	Capacity	N	int	11	N
RM7	rooms	image_url	Room's picture	N	varchar	500	N
RM8	rooms	description	Description	N	text	–	N
RM9	rooms	amenities	amenities	N	text	–	N
RM10	rooms	created_at	Created at	N	timestamp	–	Y

RM11	rooms	updated_at	Updated at	N	timestamp	–	Y

Bookings table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
BK1	bookings	id	Booking ID	Y	int	11	Y
BK2	bookings	user_id	FK → users.id	N	int	11	N
BK3	bookings	guest_name	User' name	N	varchar	100	Y
BK4	bookings	room_type	Requested Room Type	N	varchar	50	Y
BK5	bookings	room_id	FK → rooms.id	N	int	11	N
BK6	bookings	check_in_date	Checkin date	N	date	–	Y
BK7	bookings	check_out_date	Checkout date	N	date	–	N
BK8	bookings	total_price	Total price	N	decimal	10,2	Y
BK9	bookings	status	Booking status	N	enum	–	Y
BK10	bookings	special_requests	Special requests	N	text	–	N
BK11	bookings	created_at	Created at	N	timestamp	–	Y
BK12	bookings	updated_at	Updated at	N	timestamp	–	Y

Payments table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
PM1	payments	id	Payment ID	Y	int	11	Y
PM2	payments	booking_id	FK → bookings.id	N	int	11	Y
PM3	payments	amount	amount	N	decimal	10,2	Y
PM4	payments	payment_met	Payment	N	enum	–	Y

		hod	method				
PM5	payments	status	status	N	enum	–	Y
PM6	payments	transaction_id	Transaction ID	N	varchar	100	N
PM7	payments	payment_date	Payment date	N	timestamp	–	N
PM8	payments	notes	notes	N	text	–	N

Services table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
SV1	services	id	Services ID	Y	int	11	Y
SV2	services	service_name	Service name	N	varchar	100	Y
SV3	services	description	description	N	text	–	N
SV4	services	price	price	N	decimal	10,2	Y
SV5	services	category	category	N	varchar	50	Y
SV6	services	is_available	Service status	N	boolean	–	Y

Service requests table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
SR1	service_requests	id	Service requests ID	Y	int	11	Y
SR2	service_requests	booking_id	FK → bookings.id	N	int	11	Y
SR3	service_requests	service_id	FK → services.id	N	int	11	Y
SR4	service_requests	quantity	quantity	N	int	11	Y
SR5	service_requests	unit_price	Unit price	N	decimal	10,2	Y
SR6	service_requests	total_price	Total price	N	decimal	10,2	Y
SR7	service_requests	status	service status	N	enum	–	Y

Coupons table:

ID	Data	Field name	Description	Unique?	Data	Length	Req'd?

	Object				type		
CP1	coupons	id	Coupon ID	Y	int	11	Y
CP2	coupons	code	Coupon code	Y	varchar	50	Y
CP3	coupons	discount_type	coupons discount type	N	enum	–	Y
CP4	coupons	discount_value	Coupons discount value	N	decimal	10,2	Y
CP5	coupons	valid_from	Coupons valid from	N	date	–	Y
CP6	coupons	valid_to	Coupons valid to	N	date	–	Y
CP7	coupons	is_active	Coupons status	N	boolean	–	Y

Coupon usages table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
CU1	coupon_usages	id	Coupon usages ID	Y	int	11	Y
CU2	coupon_usages	coupon_id	FK → coupons.id	N	int	11	Y
CU3	coupon_usages	booking_id	FK → bookings.id	N	int	11	Y
CU4	coupon_usages	discount_amount	Coupon usages discount amount	N	decimal	10,2	Y
CU5	coupon_usages	used_at	Coupon usages used at	N	timestamp	–	Y

Checkins table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
CI	checkin	id	check-in ID	Y	int	11	Y

1	s						
CI 2	checkins	booking_id	FK → bookings.id	Y	int	11	Y
CI 3	checkins	room_id	FK → rooms.id	N	int	11	Y
CI 4	checkins	receptionist_id	FK → staff.id	N	int	11	N
CI 5	checkins	checkin_time	Checkin time	N	timestamp	–	Y

Checkouts table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
CO1	checkouts	id	Checkout ID	Y	int	11	Y
CO2	checkouts	booking_id	FK → bookings.id	Y	int	11	Y
CO3	checkouts	receptionist_id	FK → staff.id	N	int	11	N
CO4	checkouts	checkout_time	Checkout time	N	timestamp	–	Y
CO5	checkouts	total_amount	Total amount	N	decimal	10,2	Y

Wallets table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
WL1	wallets	id	Wallets ID	Y	int	11	Y
WL2	wallets	user_id	FK → users.id	Y	int	11	Y
WL3	wallets	balance	balance	N	decimal	15,2	Y

Reports table:

ID	Data Object	Field name	Description	Unique?	Data type	Length	Req'd?
RP 1	reports	id	Reports ID	Y	int	11	Y

RP 2	reports	report_type	Report type	N	enum	–	Y
RP 3	reports	generated_date	Generated date	N	timestamp	–	Y
RP 4	reports	period_start	Period start	N	date	–	N
RP 5	reports	period_end	Period end	N	date	–	N
RP 6	reports	data	Reports data	N	json	–	N
RP 7	reports	generated_by	FK → users.id	N	int	11	N
RP 8	reports	file_path	File path	N	varchar	255	N

2.2.2 Key State Models (Status Transitions)

Booking status (typical):

• pending → checked_in → checked_out

• pending → cancelled

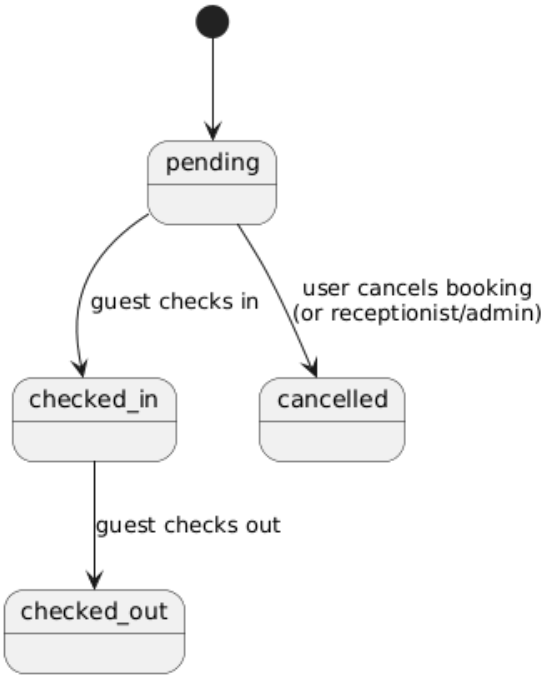

Figure 9: Key state booking status

Room status (typical):

• available → reserved → occupied → available (or dirty/maintenance depending on policy)

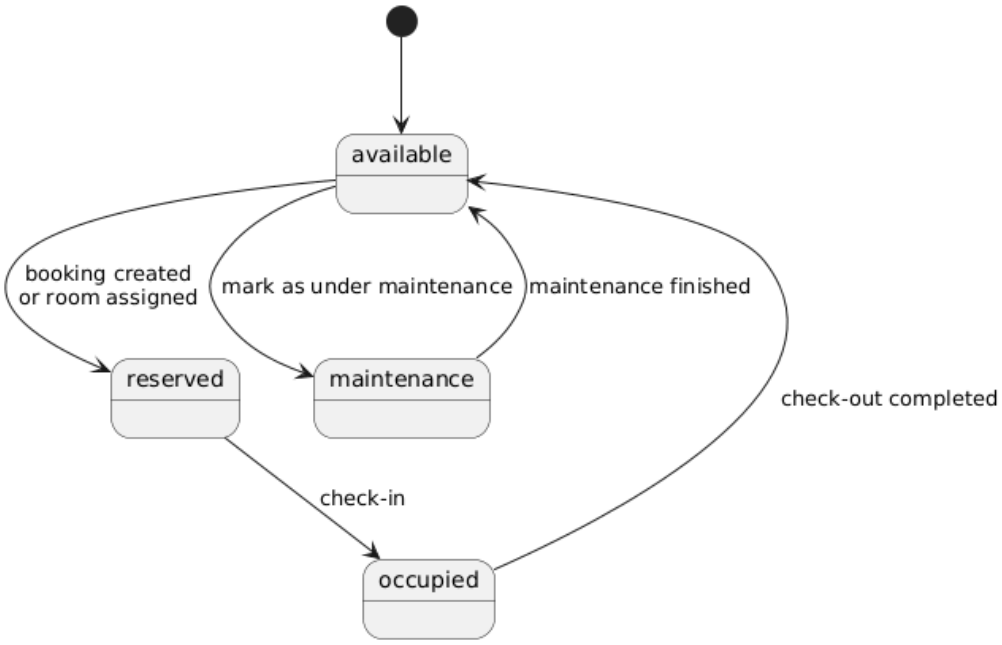

Figure 10: Key state room status

Payment status (typical):

• pending → completed → refunded (if cancellation/refund occurs)

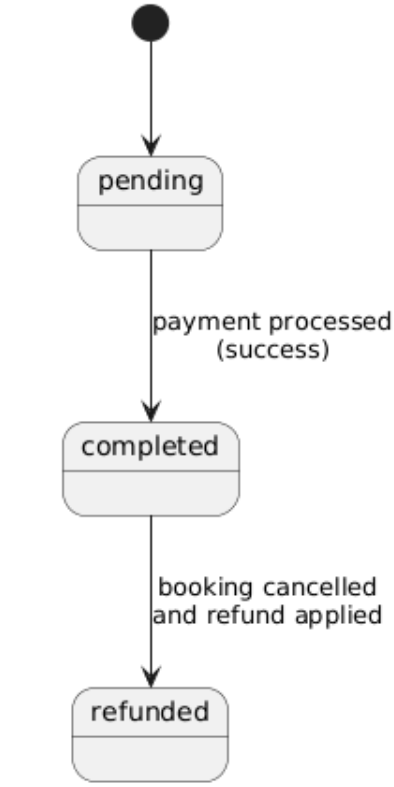

Figure 11: Key state payment status

2.2.3 API Endpoint Catalog (Core Contracts)

The following list documents the public API contracts of the monolith (grouped by domain).

Booking api:

Method	Endpoint	Auth	Role	Describe
POST	/api/bookings	Yes	user / receptionist / admin	Create booking
GET	/api/bookings/my	Yes	user / receptionist / admin	Get current user's bookings
GET	/api/bookings	Yes	receptionist / admin	Get all bookings
GET	/api/bookings/{booking_id}	Yes	user / receptionist / admin	Booking Details
PUT	/api/bookings/{booking_id}	Yes	receptionist / admin	Update Booking
DELETE	/api/bookings/{booking_id}	Yes	user / receptionist / admin	Cancel booking

Rooms api:

Method	Endpoint	Auth	Role	Describe
GET	/api/rooms	Optional	–	Room List
GET	/api/rooms/search	Optional	–	Filter Rooms by Type/Status
GET	/api/rooms/{room_id}	Optional	–	Room Details
POST	/api/rooms	Yes	admin	Create Room
PUT	/api/rooms/{room_id}	Yes	admin	Update room
DELETE	/api/rooms/{room_id}	Yes	admin	Delete room
POST	/api/rooms/assign	Yes	receptionist / admin	Assign Room
GET	/uploads/{filename}	Public	–	Return Image URL

Auth & User api:

Method	Endpoint	Auth	Role	Describe
POST	/api/auth/login	Public	–	Login
POST	/api/users	Public	–	Register
GET	/api/users	Yes	admin	User List
GET	/api/users/{user_id}	Yes	user / receptionist	User Details

			/ admin	
PUT	/api/users/{user_id}/profile	Yes	user / receptionist / admin	Update Profile
PUT	/api/users/{user_id}	Yes	admin	Admin Update User
DELETE	/api/users/{user_id}	Yes	admin	Delete User

Services & Service requests:

Method	**Endpoint**	**Auth**	**Role**	**Describe**
GET	/api/services	Optional	–	Service List
GET	/api/services/{service_id}	Optional	–	Service Details
POST	/api/services	Yes	admin	Create Service
PUT	/api/services/{service_id}	Yes	admin	Update Service
DELETE	/api/services/{service_id}	Yes	admin	Delete Service
POST	/api/services/request	Yes	user / receptionist / admin	Create Service Request
GET	/api/service-requests	Yes	receptionist / admin	Service requests list
GET	/api/service-requests/{request_id}	Yes	receptionist / admin	Request List
PUT	/api/service-requests/{request_id}	Yes	receptionist / admin	Request Details

Payments:

Method	**Endpoint**	**Auth**	**Role**	**Describe**
POST	/api/payments	Yes	user / receptionist / admin	Payments
GET	/api/payments	Yes	receptionist / admin	Transaction History
GET	/api/payments/{payment_id}	Yes	user / receptionist / admin	Payment Details
GET	/api/bookings/{booking_id}/payments	Yes	user / receptionist / admin	Payment by Booking

Coupons:

Method	Endpoint	Auth	Role	Describe
POST	/api/coupons	Yes	admin	Create coupon
GET	/api/coupons/{coupon_id}	Optional	–	Coupon detail
POST	/api/coupons/apply	Yes	user / receptionist / admin	Apply coupon

Check out:

Method	Endpoint	Auth	Role	Describe
POST	/api/checkins	Yes	receptionist / admin	Check-in
POST	/api/checkouts	Yes	receptionist / admin	Check-out
GET	/api/checkins/{checkin_id}	Yes	receptionist / admin	check-in detail
GET	/api/checkouts/summary/ {booking_id}	Yes	user / receptionist / admin	Checkout Summary

Wallet:

Method	Endpoint	Auth	Role	Describe
GET	/api/wallet	Yes	user / receptionist / admin	wallet
POST	/api/wallet/topup	Yes	user / receptionist / admin	Top-up Wallet

Staff:

Method	Endpoint	Auth	Role	Describe
GET	/api/staff	Yes	admin	Staff list
POST	/api/staff	Yes	admin	Create staff
GET	/api/staff/{staff_id}	Yes	admin	Staff detail
PUT	/api/staff/{staff_id}	Yes	admin	Update staff
DELETE	/api/staff/{staff_id}	Yes	admin	Delete staff

Reports:

Method	Endpoint	Auth	Role	Describe
POST	/api/reports/revenue	Yes	admin	Revenue Report
POST	/api/reports/occupancy	Yes	admin	Occupancy Report
POST	/api/reports/booking	Yes	admin	Booking Report
GET	/api/reports/{report_id}	Yes	admin	Report detail
GET	/api/reports/type/{report_type}	Yes	admin	Report by Category

2.2.4 Authorization Matrix (Role-based Access Control Summary)

- Public (no auth): /api/auth/login, /api/users (register), /uploads/*
- Optional auth (guest can access): GET /api/rooms*, GET /api/services*, GET /api/coupons/<id>

- User: booking create/cancel/view own, payments by booking, wallet, request service, apply coupon
- Receptionist: booking list/update, assign room, check-in/check-out, manage service requests, payment list
- Administrator: all receptionist permissions + full CRUD for rooms/services/users/staff + reports + coupons

2.3 Implementation: Server-Side Logic (Flask Backend – Controllers/Services)

Example: Booking Creation Flow
1. The frontend (e.g., from rooms.html or room-detail.html) sends a POST request to /api/bookings with JSON body containing:
- guest_name
- room_type
- check_in_date (and optionally check_out_date)
- optional total_price override
- optional payment_method and payment_amount
2. app.py defines the route:
- @app.route('/api/bookings', methods=['POST']) @require_auth def create_booking(): return booking_controller.create_booking()
3. BookingController.create_booking() reads request.json, obtains the current user from request.current_user (set by the authentication middleware), and calls BookingService.create_booking(...).
4. BookingService.create_booking() performs:
- Input validation (guest name, room type, dates).
- Date parsing and validation (no past check-in, check-out after check-in).
- Room availability check by calling RoomService.search_rooms(..., check_in_date, check_out_date). This uses BookingRepository.find_available_rooms_by_date_range() to exclude rooms with overlapping bookings.
- Room selection (first available) and status update to "reserved" via RoomRepository.
- Booking creation via BookingRepository.save(...) within a transaction.
- Optional payment processing by calling PaymentService.process_payment(...) if payment details were provided. If payment fails, the service releases the room and cancels the booking.
5. BookingController then uses BookingView.booking_created(booking) to format a JSON response including booking details for the frontend.

Example: Booking Cancellation & Refund Flow
1. The frontend sends DELETE /api/bookings/<booking_id>, which is protected by @require_auth in app.py.
2. BookingController.cancel_booking(booking_id) calls BookingService.cancel_booking.
3. BookingService.cancel_booking:

- Validates that the booking exists and is not already checked_in, checked_out, or cancelled.
- Ensures there is no check-in record (via CheckInRepository).
- Releases the room by setting its status back to "available" if it was "reserved".
- Looks up completed payments for the booking through PaymentService and, if applicable, refunds the amount to the user's wallet and updates payment status to "refunded".
- Updates booking status to "cancelled" via BookingRepository.delete (which sets status = 'cancelled').

4. A descriptive message is returned to the client, including refund notes if any.

Example: Coupon Application Flow
1. The frontend sends POST /api/coupons/apply with a coupon code.
2. CouponController.apply_coupon() calls CouponService.apply_coupon(code).
3. CouponService.apply_coupon:
- Loads the coupon by code via CouponRepository.
- Validates that the coupon exists and is active.
- Checks current date against coupon.valid_from and coupon.valid_to, ensuring The coupon is neither expired nor not yet valid.
- Returns coupon details to the controller, which then responds with discount information so that the frontend can adjust the total price.

Example: Payment Processing Flow
1. Frontend sends POST /api/payments with JSON:
- booking_id, amount, payment_method (credit_card/cash/bank_transfer), optional user_id context.
2. PaymentController.process_payment delegates to PaymentService.process_payment(...).
3. PaymentService validates:
- amount > 0; payment_method in allowed methods; booking exists and not cancelled.
4. If wallet-based payment is used in the current design, the service checks wallet balance and deducts.
5. PaymentRepository saves payment records (status pending/completed).
6. Backend returns payment result to the frontend.

Example: Check-in Flow (Receptionist)
1. The receptionist selects a booking and calls POST /api/checkins (booking_id, receptionist_id, optional room_id).
2. CheckInController delegates to CheckInService.process_checkin(...).
3. Service validates booking exists, not already checked in, and room assignment matches booking.room_type.
4. Repository saves check-in records and service updates booking status to checked_in and room status to occupied.

5. Backend returns the check-in record.

Example: Check-out Flow (Receptionist)
1. The receptionist calls POST /api/checkouts with booking_id and checkout details.
2. CheckInService validates that a check-in exists for the booking.
3. Service calculates total checkout amount:
- room cost (booking.total_price) + additional services (service_requests total).
4. Repository saves checkout record; service updates booking status to checked_out and releases/updates room status.
5. Backend returns checkout result; user can view summary via GET /api/checkouts/summary/<booking_id>.

Example: Service Request Flow
1. User requests a service via POST /api/services/request (booking_id, service_id, quantity).
2. ServiceController delegates to ServiceService.create_service_request(...).
3. ServiceService validates booking exists and service exists, then creates service_requests record.
4. Receptionist/Admin can view and manage requests via:
- GET /api/service-requests, GET /api/service-requests/<id>, PUT /api/service-requests/<id>.

Example: Wallet Flow
1. User opens wallet page → GET /api/wallet.
2. WalletService returns wallet; if missing, creates one with balance 0.
3. User tops up → POST /api/wallet/topup with amount; system validates numeric and > 0, then updates balance.

2.3.1 Key Validation & Error Handling Strategy

Input validation is primarily implemented in the Service layer (e.g., BookingService, PaymentService, CouponService):
- Date validation: YYYY-MM-DD; check-in not in past; check-out after check-in.
- Money validation: amount > 0; discounts within valid ranges.
- State validation: cannot cancel/update after check-in; cannot checkout without check-in.
- Role validation: enforced by middleware decorators (401/403).

Error Response Strategy (high-level):
- 400 Bad Request: invalid input data, business rule violation
- 401 Unauthorized: authentication required / missing X-User-Id
- 403 Forbidden: role not permitted

- 404 Not Found: resource not found (booking/room/service/coupon/etc.)
- 500 Internal Server Error: database or unexpected server errors

2.3.2 Runtime/Deployment View

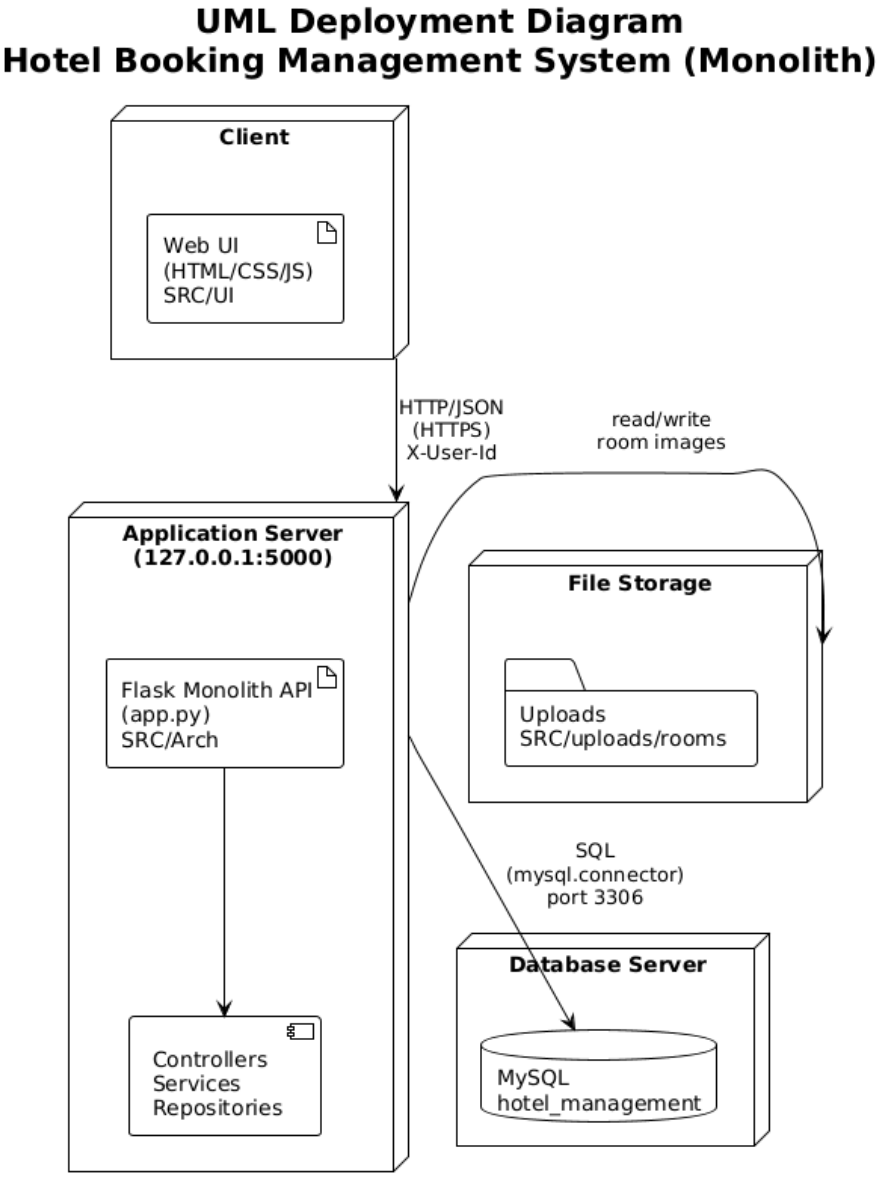

Figure 12: Deployment View

- The monolith runs as a single Flask process (default: 127.0.0.1:5000).
- The MySQL database is a separate process/service configured by environment variables in .env.
- UI files under SRC/UI are static and call APIs under /api/* via JavaScript (fetch).
- Uploaded images are stored under SRC/uploads and served through /uploads/<path>.

2.4 Implementation: Client-Side Logic (Web UI – HTML/CSS/JS)

The UI in SRC/UI is separated by role and page:
Public & User pages:

- index.html: homepage introducing the hotel and quick access to room browsing.
- public/rooms.html and public/room-detail.html: room list and details pages.
- user/dashboard.html: user overview, showing relevant booking and wallet info.
- user/my-bookings.html: history and status of user's bookings.
- user/payment.html: payment UI for a specific booking.
- user/wallet.html: wallet balance, top-up form, and transaction history view (if implemented).

Authentication pages:

- auth/login.html and auth/register.html: forms that send login/registration requests to /api/auth/login and /api/users.

Staff & Admin pages:

- admin/admin.html: admin dashboard for managing rooms, staff, services, coupons, and viewing reports.
- admin/receptionist.html: interface tailored to receptionist tasks like managing arrivals, check-ins, check-outs, and service requests.
- admin/admin-messages.html: page for internal messages or notifications.

JavaScript files (SRC/UI/js):

- api.js: centralizes AJAX/fetch calls to backend APIs; likely wraps GET/POST/PUT/DELETE requests for rooms, bookings, payments, coupons, wallet, etc.
- auth.js: handles storing auth tokens, sending login/register forms, and updating the UI based on user authentication state.
- admin.js and receptionist.js: contain logic to populate admin/receptionist tables, call backend endpoints (/api/rooms, /api/service-requests, /api/staff, /api/reports), and react to user actions (create room, accept service request, etc.).
- script.js and language.js: implement shared UI functionalities (menu behavior, language switching), while admin-messages.js and chat.js support additional user/admin communication features.